

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И
ИНФОРМАТИКИ

Кафедра «Программная инженерия»

Лабораторная работа № 4
по дисциплине «Введение в информационные технологии»
по теме: «Разработка RESTful API: спецификация и архитектура»

Выполнил студент группы:

БВТ2504

Гапеев Алексей Денисович

Проверил: Харрасов К.Р.

Москва 2026

Оглавление

Цель работы.....	3
Задание 1. Назначение HTTP-статус-кодов.....	4
Задание 2. Валидация входных данных.....	7
Задание 3. Обработка ошибок.....	9
Задание 4. Path- и query-параметры.....	10
Задание 5. Тестирование через Postman.....	11
Итоговое практическое задание.....	12
Вывод.....	16

Цель работы:

Цель работы: Получить практические навыки в разработке RESTfull API и использовании swagger.

Задание 1. Назначение HTTP-статус-кодов

Модифицируйте существующие endpoints таким образом, чтобы каждому из них был назначен корректный HTTP-статус ответа:

- GET — 200 OK
- POST — 201 Created

Проверьте работу endpoints через Swagger и Postman.

```
@app.get("/", status_code=200, description="OK")
def joke():
    return pyjokes.get_joke()

@app.get("/{friend}", status_code=200, description="OK")
def joke_from_friend(friend: str):
    return f'{friend} say his joke: ' + pyjokes.get_joke()

@app.get("/multi/{friend}", status_code=200, description="OK")
def multi_jokes_from_friend(friend: str, jokes_numbers: int):
    result_str = ""
    for i in range(jokes_numbers):
        result_str += " " + friend + f'say his joke №{i + 1}: ' + pyjokes.get_joke() + " "
    return result_str

class Joke(BaseModel):
    friend: str
    joke: str

class JokeInput(BaseModel):
    friend: str

@app.post("/", status_code=201, description="Created")
def create_joke(joke_input: JokeInput):
    return Joke(friend=joke_input.friend, joke=pyjokes.get_joke())
```

Текст 1: Код программы для задания 1

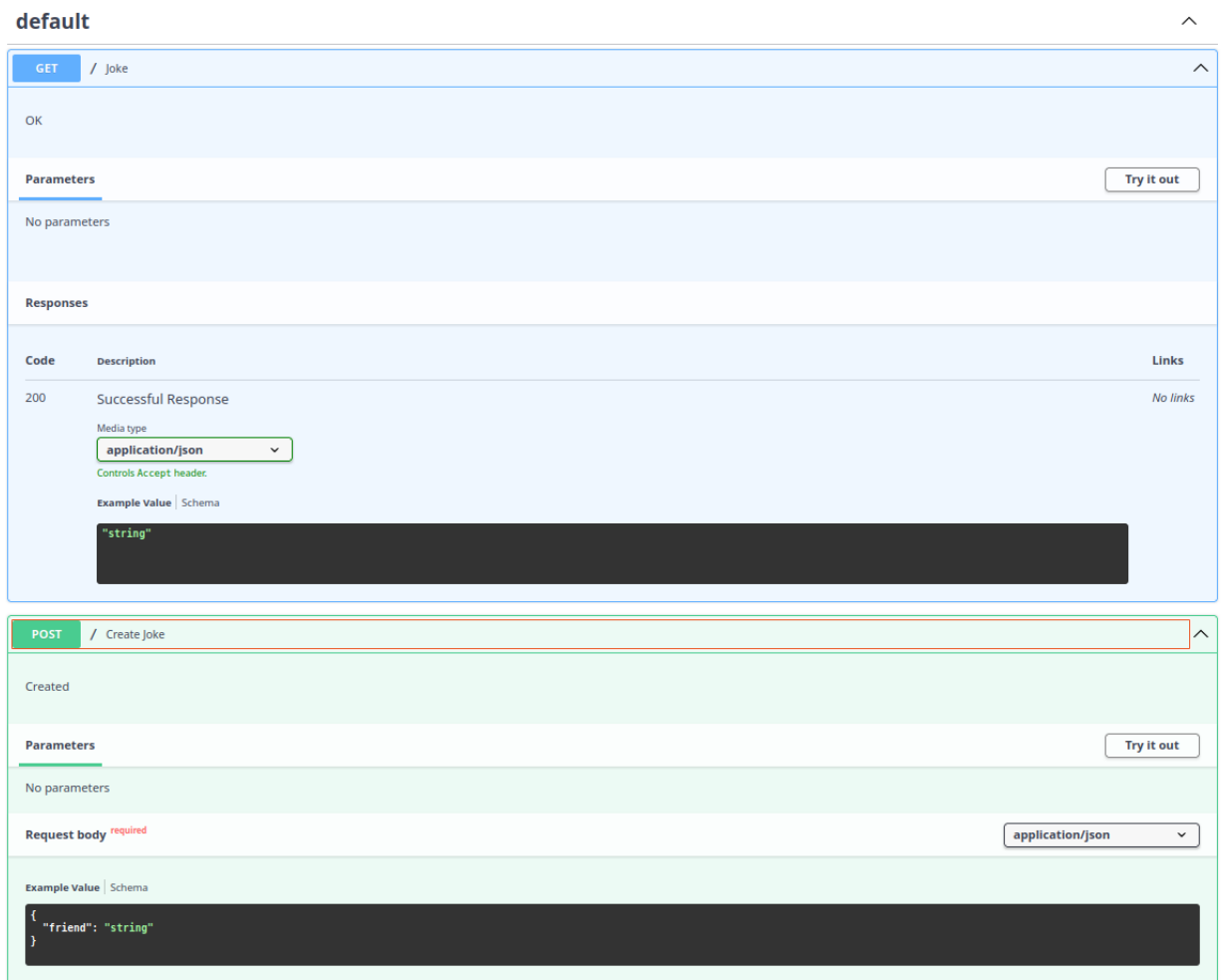


Рисунок 1: Страница Swagger



Рисунок 2: GET запрос в Postman

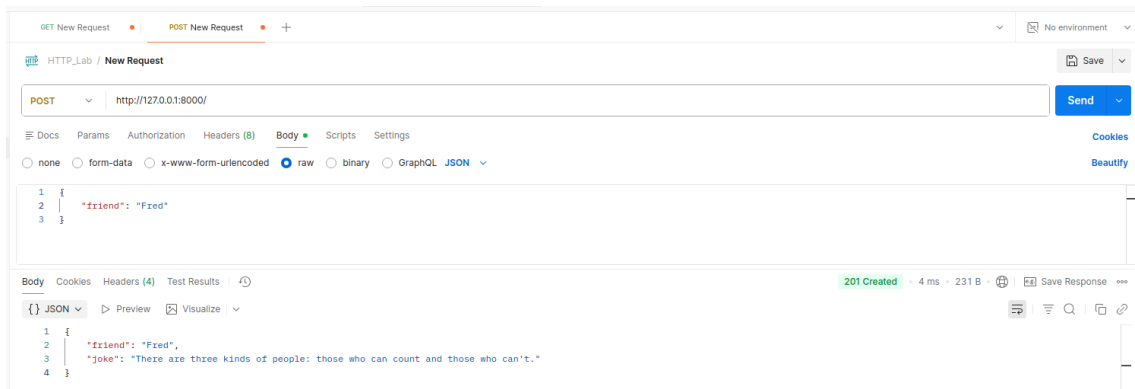


Рисунок 3: POST запрос в Postman

Задание 2. Валидация входных данных

Добавьте ограничения на входные параметры с использованием Pydantic: минимальную длину строки для параметра friend, допустимый диапазон значений для параметра jokes_number.

Передайте некорректные данные и зафиксируйте ответ сервера. Объясните появление ошибки 422 Unprocessable Entity.

```
class Multijokes(BaseModel):
    friend: str = Field(min_length=3)
    joke_number: int = Field(gt=0, le=15)

@app.post("/testing_joke", status_code=201, description="Created")
def testing_joke(joke_input: Multijokes):
    result = ""
    for i in range(joke_input.joke_number):
        result += joke_input.friend + f" tells his joke № {i + 1}" + ": " + pyjokes.get_joke()
    return result
```

Текст 2: Модифицированный код для задания 2

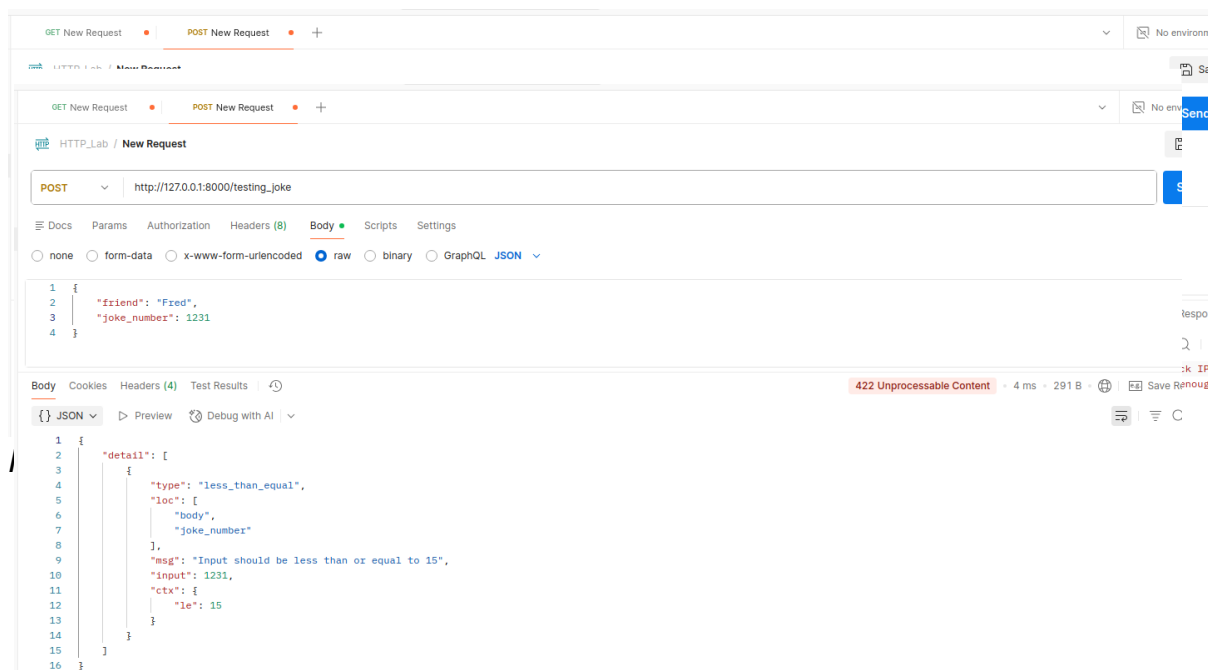


Рисунок 5: Некорректные данные и ошибка 422

Ошибка 422 Unprocessable Content означает, что сервер принял и распознал входные данные, но не может выполнить инструкции из-за несоответствия правилам валидации. В данном случае, входной параметр `joke_number = 15` не соответствует условию `0 < joke number < 15`.

Задание 3. Обработка ошибок

Реализуйте проверку параметра `jokes_number`. В случае передачи недопустимого значения верните ошибку 400 Bad Request с пояснением причины. Проверьте работу эндпоинта при корректных и некорректных данных.

```
@app.post("/multi", status_code=201, description="Created")
def multi_jokes_from_friend(joke_input: JokeInput):
    result_str = ""
    if joke_input.jokes_number > 5:
        raise HTTPException(status_code=400, detail="Too many jokes.")
    for i in range(joke_input.jokes_number):
        result_str += " " + joke_input.friend + f"say his joke №{i + 1}: " + pyjokes.get_joke() + " "
    return result_str
```

Текст 3: Код для задания 3

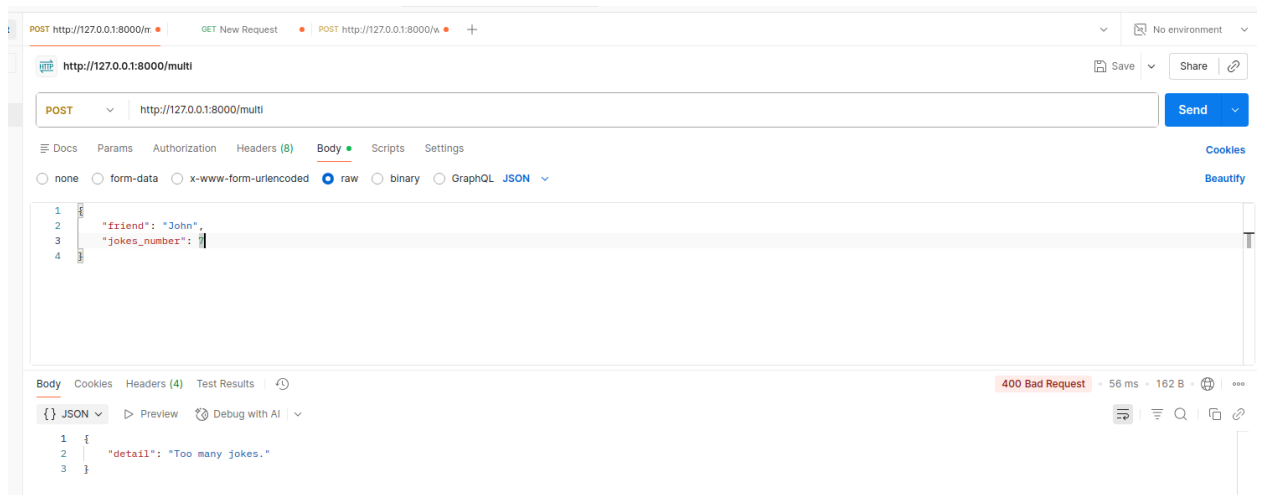


Рисунок 6: Ответ сервера

Задание 4. Path- и query-параметры

Реализуйте два эндпоинта с одинаковой логикой:

- один с использованием path-параметра
- один с использованием query-параметра

Сравните способы передачи параметров и сделайте вывод о различиях между ними.

```
@app.get("/something1/{text}")
def something(text: str):
    return text
```

```
class Something(BaseModel):
    text: str
```

```
@app.get("/something2")
def something_arr(something3: Something = Depends()):
    return something3.text
```

Текст 4: Код к заданию 4

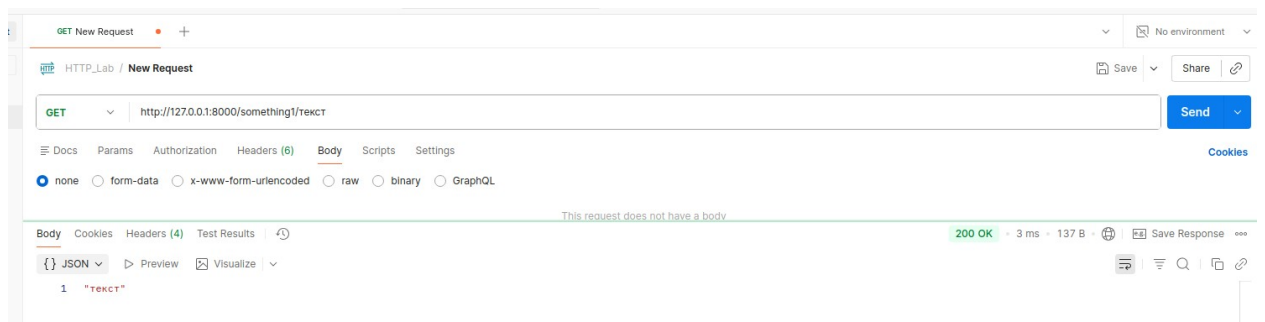


Рисунок 7: Path-параметр

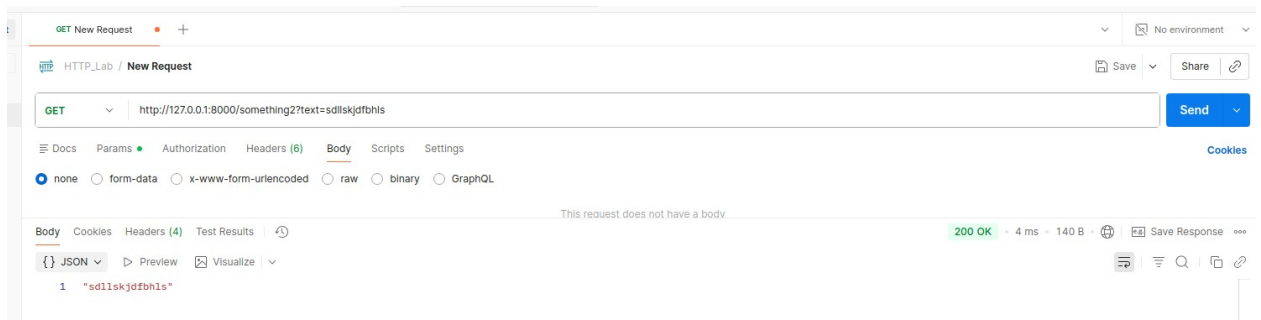


Рисунок 8: Query-параметр

Задание 5. Тестирование через Postman

Протестируйте все реализованные эндпоинты с использованием Postman. Для каждого запроса:

- зафиксируйте метод и URL
- укажите HTTP-статус ответа
- сохраните фрагмент ответа сервера

В отчёт включите скриншоты корректных и ошибочных запросов.

Итоговое практическое задание

Разработать простой REST-сервис с использованием библиотеки wikipedia. Сервис должен предоставлять доступ к информации из Wikipedia через HTTP-запросы. Для реализации использовать фреймворк FastAPI. Все эндпоинты должны возвращать ответы в формате JSON и валидироваться с помощью схем Pydantic. Ссылка на документацию по Wikipedia в python https://rukovodstvo.net/posts/id_1061/

1. Эндпоинт с path-параметром

Реализовать эндпоинт, принимающий название статьи Wikipedia в качестве path-параметра.

Пример запроса:

```
GET /wiki/path/{title}
```

Эндпоинт должен:

- получать краткое описание статьи из Wikipedia
- возвращать результат в виде JSON
- использовать схему ответа

2. Эндпоинт с query-параметром

Реализовать эндпоинт, принимающий название статьи Wikipedia в качестве query-параметра.

Пример запроса:

```
GET /wiki/query?title=Python
```

Эндпоинт должен:

- выполнять тот же функционал, что и предыдущий
- принимать параметр через строку запроса
- возвращать результат по схеме

3. Эндпоинт с передачей данных в теле запроса

Реализовать эндпоинт, принимающий название статьи Wikipedia через тело запроса (POST).

Пример запроса:

```
POST /wiki/body
```

Тело запроса (JSON):

```
{  
"title": "Python"  
}
```

Эндпоинт должен:

- принимать данные через Pydantic-схему
- возвращать структурированный ответ
- валидировать входные данные

```
from fastapi import FastAPI, Depends  
from pydantic import BaseModel, Field  
import wikipedia
```

```
app = FastAPI()
```

```
class Title(BaseModel):  
    title: str = Field(min_length=2)
```

```
@app.get("/")  
def welcome():  
    return "Welcome Page"
```

```
# Пункт 1.  
@app.get("/wiki/path/{title}")  
def summary(title: Title = Depends()):  
    return wikipedia.summary(title.title)
```

```
# Пункт 2.  
@app.get("/wiki/query")  
def summary(title: Title = Depends()):  
    return wikipedia.summary(title.title)
```

```
# Пункт 3.  
  
@app.post("/wiki/body")  
def body(title: Title):  
    return wikipedia.summary(title.title)
```

Текст 5: Код программы

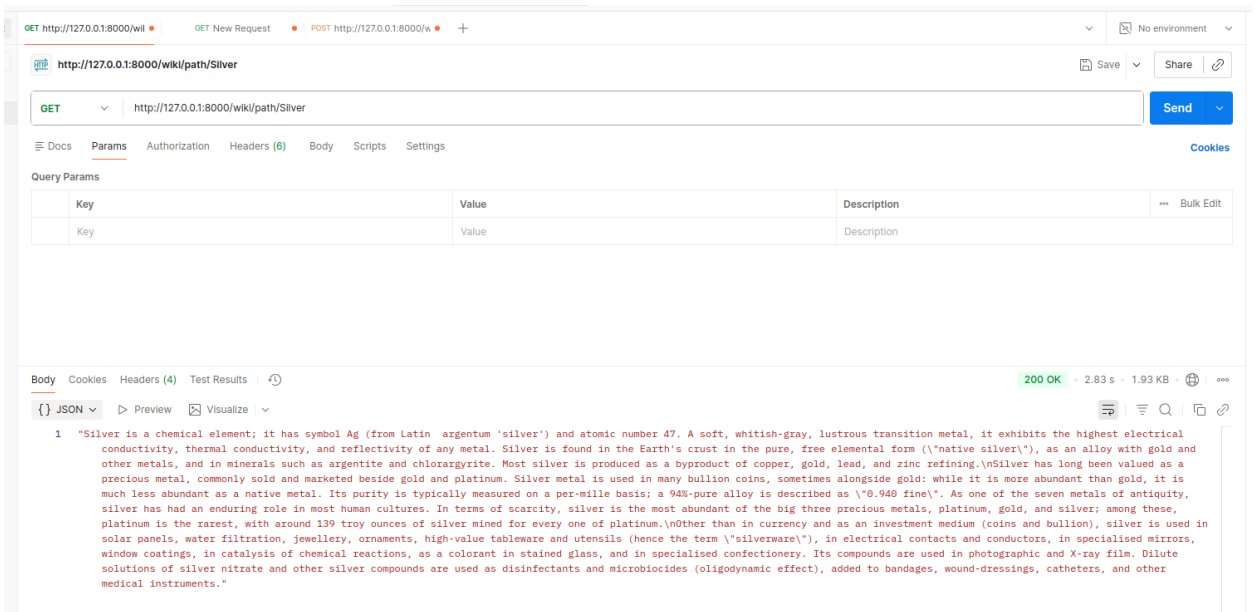


Рисунок 9: Path параметр

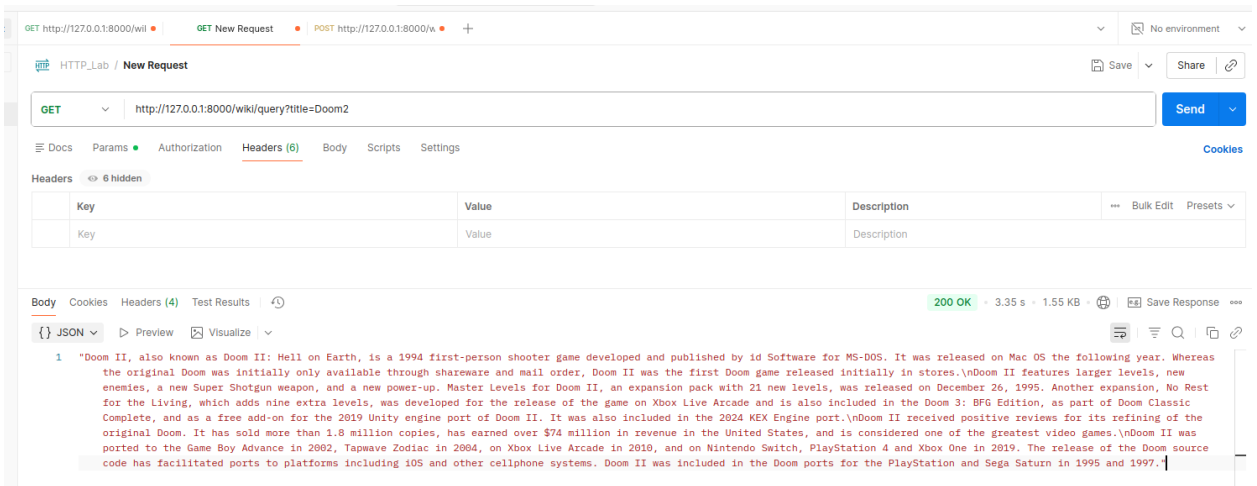
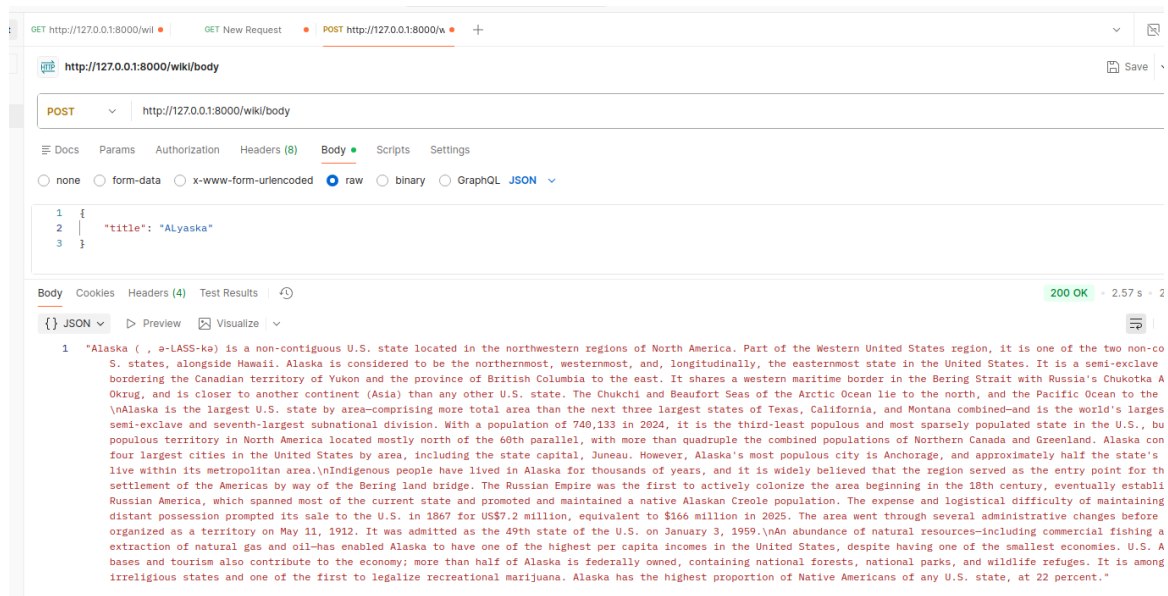


Рисунок 10: Query параметр



Вывод

В результате выполнения лабораторной работы были освоены навыки работы Path- и Query параметрами, изучены библиотеки FastAPI и Pydantic. Разработан собственный API по индивидуальному заданию.